

INTUITION_OS_MASTER_SPEC.md

PART 3

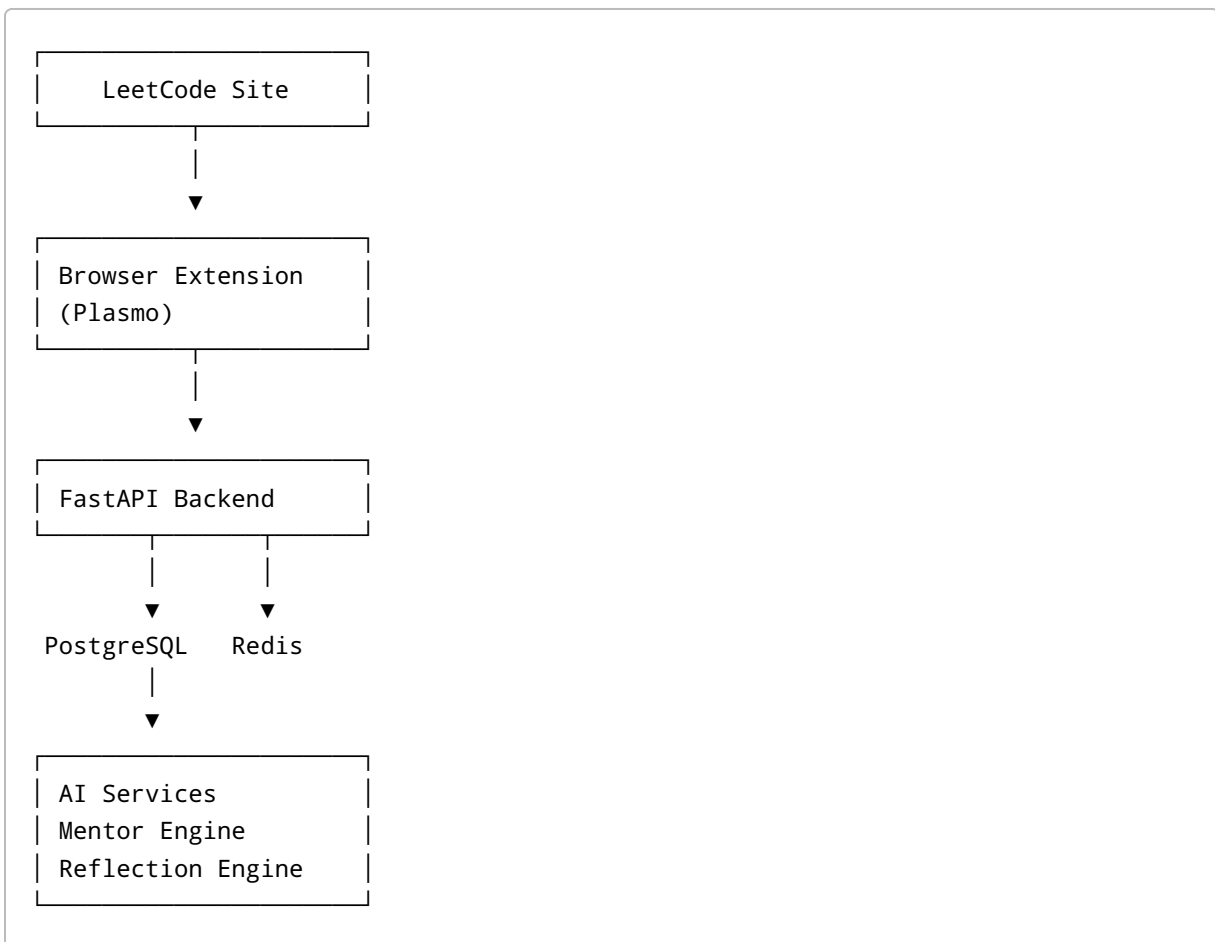
TECHNICAL ARCHITECTURE

EXTENSION · BACKEND · DATABASE · AI SYSTEM

Version 1.0

System Overview

Intuition OS consists of five primary systems:



Architecture Principles

Principle 1

Keep V1 extremely simple.

Do not overengineer.

Principle 2

Learning Session is the core object.

Every system revolves around Learning Sessions.

Principle 3

Evidence is stored.

Scores are generated.

Never store scores directly.

Principle 4

The AI should remain stateless.

Learning memory lives in the database.

Context assembly injects memory into prompts.

Technology Stack

Extension

Framework:

Plasmo

Language:

TypeScript

UI:

React

State:

Zustand

Styling:

TailwindCSS

Dashboard

Framework:

Next.js 15 App Router

Language:

TypeScript

Styling:

TailwindCSS

Components:

shadcn/ui

State:

Zustand

Data Fetching:

TanStack Query

Forms:

React Hook Form

Validation:

Zod

Backend

Framework:

FastAPI

Language:

Python 3.12

ORM:

SQLAlchemy 2

Migrations:

Alembic

Validation:

Pydantic

Background Tasks:

Celery (future)

Authentication:

JWT

Database

PostgreSQL

Primary storage.

Cache

Redis

Used for:

- Session cache
 - Prompt cache
 - Rate limiting
-

AI Providers

V1:

- Claude
- GPT
- Gemini

Provider abstraction required.

Must allow swapping models.

Browser Extension Architecture

Extension Responsibilities

The extension should only:

1. Detect problems
2. Track sessions
3. Open mentor UI
4. Send events

Nothing else.

No code execution.

No editor replacement.

No custom IDE.

Extension Modules

Content Script

Injected into LeetCode.

Responsibilities:

Extract:

- Problem title
- URL
- Difficulty
- Tags

Detect:

- Editorial opened
 - Submission results
-

Background Service

Responsibilities:

- Authentication state
 - API communication
 - Session lifecycle
-

Popup UI

Responsibilities:

Display:

- Session information
 - Mentor entry point
 - Recent mentor responses
-

Mentor Widget

Embedded panel.

Allows:

- Request help
 - Continue conversation
 - View reflections
-

Session Lifecycle

Session Start

Triggered when:

User opens problem page.

System creates:

Learning Session.

Store:

- Problem metadata
 - Timestamp
-

Session Active

Track:

- Time spent
 - Submission events
 - Mentor interactions
-

Session End

Triggered by:

- Accepted solution
- User exits
- Timeout

Reflection generated.

Evidence extracted.

Profile updated.

Dashboard Architecture

Dashboard Purpose

Dashboard is not analytics.

Dashboard is a learning operating system.

Homepage

Sections:

Mentor Summary

Current focus.

Current bottleneck.

Recommendations.

Strengths

Evidence-backed strengths.

Weaknesses

Evidence-backed weaknesses.

Recent Growth

Recent improvements.

Recommended Problems

Personalized learning plan.

Session History Page

Displays:

Learning Sessions.

For each session:

- Problem
 - Reflection
 - Evidence
 - Mentor interactions
-

Growth Page

Displays:

Topic evolution.

Behavior evolution.

Learning trends.

Backend Architecture

Service Layer

Auth Service

Handles:

- Google OAuth
 - JWT generation
 - User creation
-

Session Service

Handles:

- Session creation
 - Session updates
 - Session closure
-

Mentor Service

Handles:

- Context assembly
 - Prompt generation
 - LLM communication
-

Reflection Service

Handles:

- Reflection generation
 - Evidence extraction
-

Profile Service

Handles:

- User summaries
- Learning history

Database Design

USERS

```
id UUID PK
email
name
avatar_url
created_at
updated_at
```

SESSIONS

```
id UUID PK
user_id FK
problem_title
problem_url
difficulty
tags JSONB
status
started_at
ended_at
duration_seconds
```

SUBMISSIONS

```
id UUID PK
session_id FK
result
created_at
```

Result values:

- WA
- TLE
- RE
- MLE
- AC

MENTOR_INTERACTIONS

```
id UUID PK
session_id FK
level
question
user_response
mentor_response
created_at
```

REFLECTIONS

```
id UUID PK
session_id FK
```

content
created_at

EVIDENCE

id UUID PK
user_id FK
session_id FK
evidence_type
details JSONB
confidence
created_at

Evidence Types

Examples:

SOLVED_WITHOUT_HINT
SOLVED_WITHOUT_EDITORIAL
RECOVERED_FROM_TLE
RECOVERED_FROM_WA
NEEDED_HINT_LEVEL_1
NEEDED_HINT_LEVEL_2
PATTERN_RECOGNIZED
PATTERN_MISSED

OPTIMIZATION_DISCOVERED

API Design

Base:

/api/v1

Auth

POST /auth/google

GET /auth/me

Sessions

POST /sessions

GET /sessions

GET /sessions/{id}

PATCH /sessions/{id}

POST /sessions/{id}/close

Mentor

POST /mentor/help

POST /mentor/respond

Reflections

`GET /reflections/{sessionId}`

Profile

`GET /profile`

`GET /profile/strengths`

`GET /profile/weaknesses`

AI Architecture

Mentor Pipeline

Input:

Current Problem

Current Session

Current Interaction

User History

Evidence

Context Assembly

The most important AI component.

Context Priority

1. Current Problem
 2. Current Session
 3. Current Mentor Interaction
 4. Similar Historical Problems
 5. User Evidence
 6. User Profile
-

Context Window Budget

Most important information first.

Never overload prompts.

Mentor Prompt Structure

System Prompt

↓

User Context

↓

Current Session

↓

Current Question

↓

Escalation Level

↓

Output

Mentor Output Format

JSON

```
{
  "observation": "",
  "question": "",
  "guidance": "",
  "escalation_level": 1
}
```

Reflection Pipeline

Session Ends

↓

Gather Data

↓

Build Reflection Prompt

↓

Generate Reflection

↓

Generate Evidence

↓

Store

↓

Update Profile

Evidence Extraction Pipeline

Reflection

↓

Evidence Extractor

↓

Evidence Records

↓

Profile Update

Profile Generation

Never stored permanently.

Generated dynamically.

Based on evidence.

This prevents profile drift.

Security

Authentication

Google OAuth only.

JWT access tokens.

Refresh tokens.

Data Privacy

Store:

Learning behavior.

Do not store:

Sensitive personal information.

AI Safety

Mentor must never:

- Auto-generate final code
 - Skip escalation
 - Reveal solution immediately
-

Scalability

V1

Single backend.

Single database.

Single Redis instance.

V2

Horizontal API scaling.

Read replicas.

Async reflection generation.

V3

Dedicated AI orchestration service.

Dedicated recommendation service.

Non Goals

Claude MUST NOT build:

- Online IDE
- Code execution
- Multiplayer collaboration
- Social network
- Discussion forum
- XP systems
- Gamification
- Contest platform
- Rating system

These are intentionally excluded.

Definition of MVP Success

A user should be able to:

1. Install extension
2. Login with Google
3. Open LeetCode
4. Start session
5. Request mentor help
6. Receive contextual guidance
7. Solve problem
8. Receive reflection
9. View learning history

If this works reliably, MVP is successful.

END OF PART 3